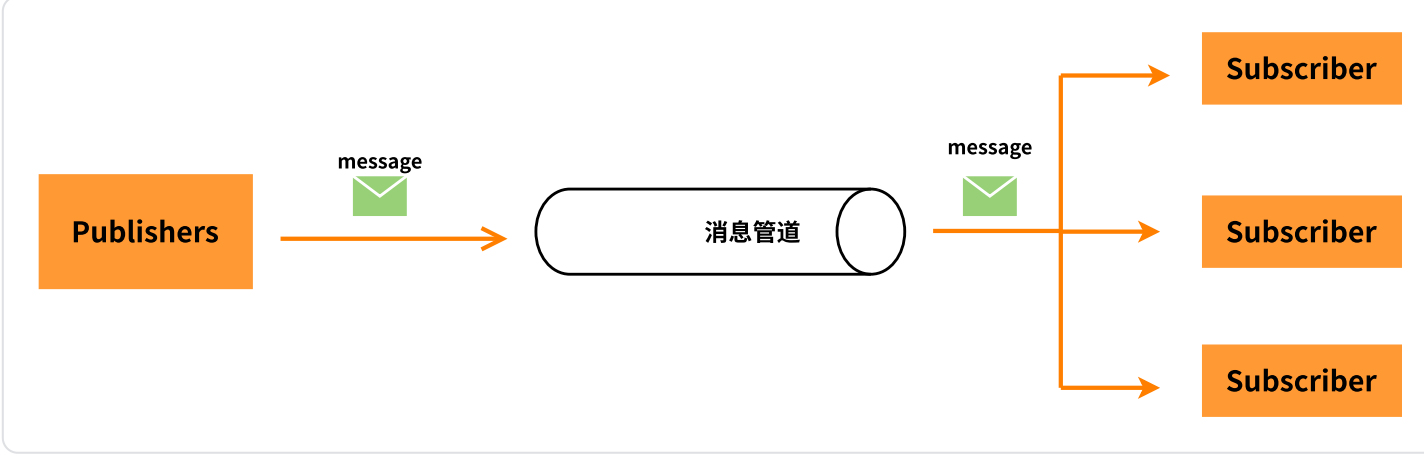
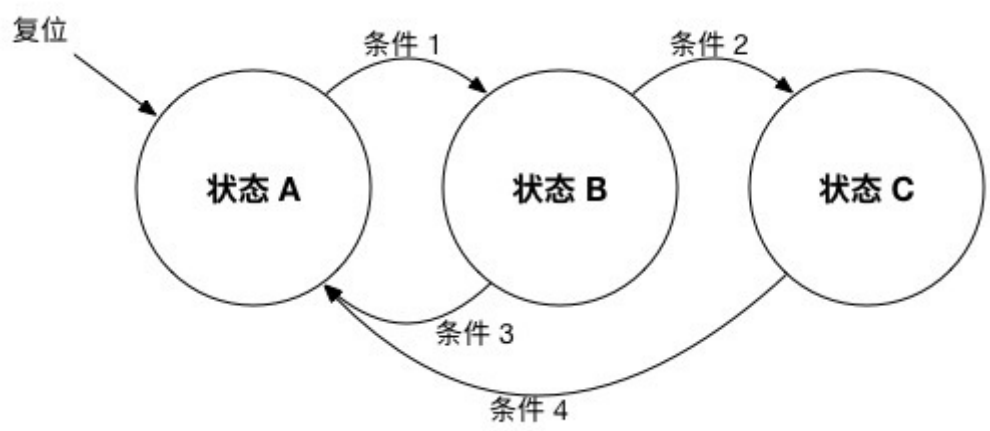
tinyToolkit

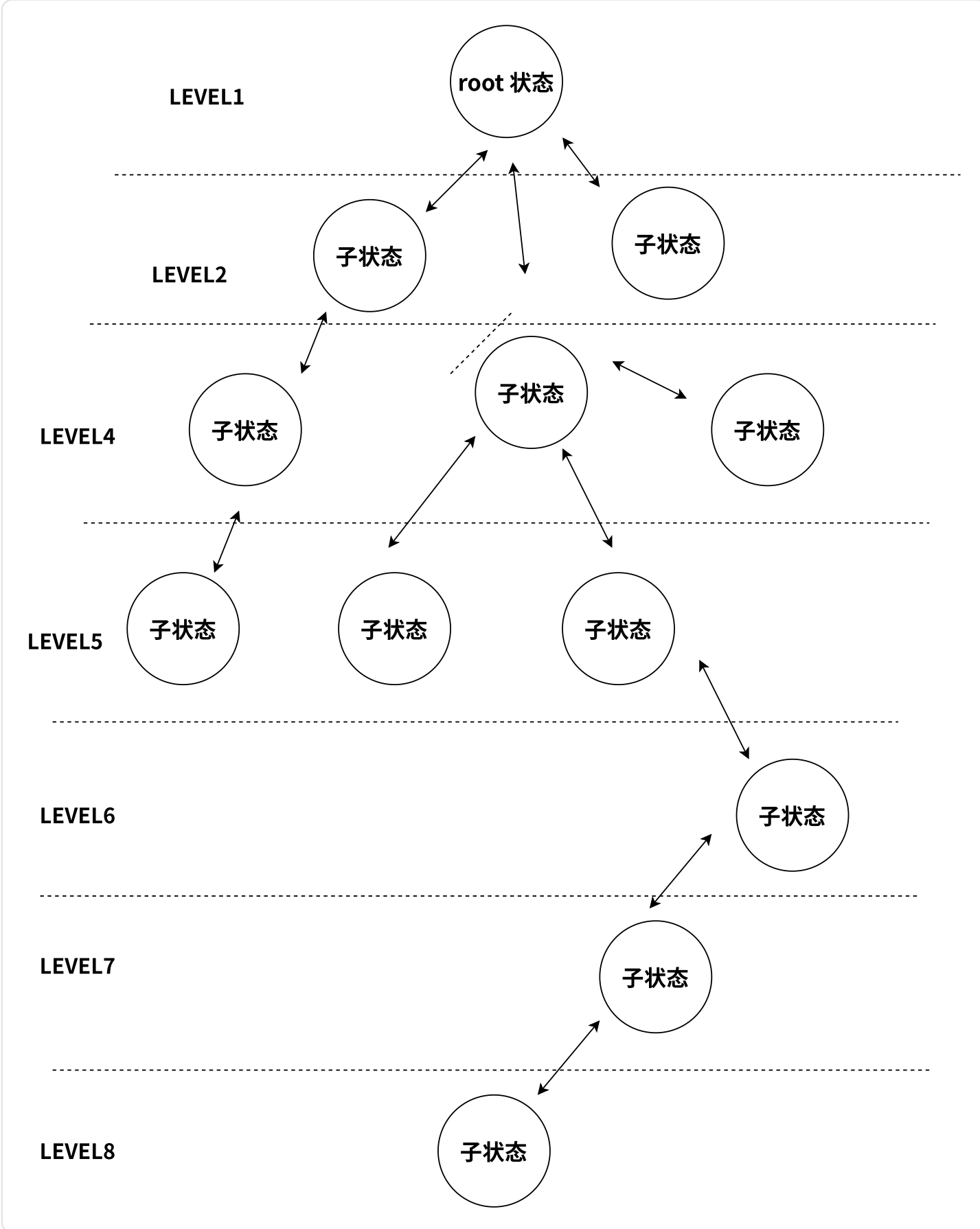
1.订阅发布框架

```
Publisher:            投递消息至总线
Subscriber:           订阅消息
消息管道:             总线,安全消息队列,可替换为高性能无锁队列
message:              任意消息类型,继承基类
Publisher-Subscriber 对于每个订阅者本身,底层为多生产在单消费者
```



2.状态机





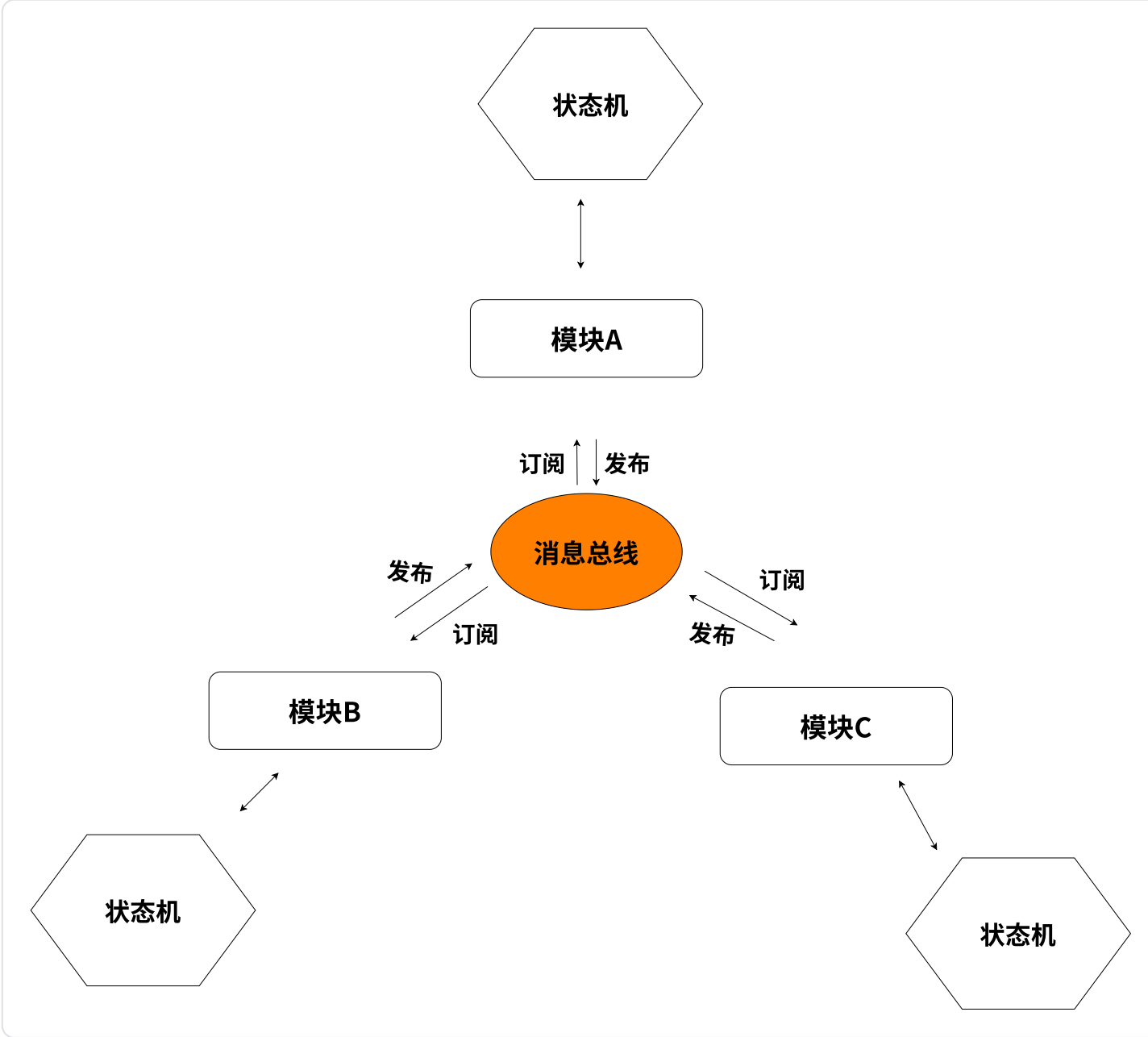
1. 架构说明

```
1>状态数据结构采用"树" 数据结构,无子节点长度深度限制
2>初始化是必须初始化跟节点,且根节点上一届节点必须为nullpet
3>每次子状态有且必须有一个父节点
4>节点必须要包含一个level字段
5>节点之间的跳转必须有且只有一条线路
6>到达公共节点之前必须先退出,然后在进入
```

```
/*
*     核心接口
*/

///statemachine.h .cpp
 class StateMachine
```

```
    {
    public:
        StateMachine();

        ~StateMachine();

    public:
        /**
         * 初始根状态
         * */
        void Init(InterfaceFuncNode rootFuncNode);

        /**
         * 添加节点,与其他节点,root节点组成状态树
         * */
        void AddNode(InterfaceFuncNode funcNode);

        /**
         *  状态切换
         * */
        void JumpNode(InterfaceFuncNode funcNode);

    private:
        void EnterStatueNode(StatusNode *dst);

    public:
        virtual void Dispatch(std::shared_ptr<NodeEvent> e);

    private:
        StatusNode *_current{nullptr}; ///当前状态节点
        std::map<InterfaceFuncNode, StatusNode> _sateMap{};///状态表
    };
```

3.订阅发布状态机框架



1. 消息接口

```
   ///event.h
    typedef unsigned int NodeSignal;

    typedef struct NodeEvent
    {
        explicit NodeEvent(NodeSignal sig) : _sig(sig)
        {}

        NodeSignal _sig;
    } NodeEvent;


///订阅发布接口
/// active.h
/// actor.h

class Actor;

    class Active : public StateMachine
    {
    public:
        Active(std::shared_ptr<Actor> actor) : _actor(actor)
        {

        }

```

```
        ~Active() = default;

    public:

        virtual void Stop();

        virtual void Start();

        virtual void PushBack(std::shared_ptr<NodeEvent> e);

        virtual void PushFront(std::shared_ptr<NodeEvent> e);

        virtual void Subscribe(NodeSignal event);

        virtual void Unsubscribe(NodeSignal event);

        virtual void Broadcast(std::shared_ptr<NodeEvent> e);

    private:
        std::thread _runThread;
        std::atomic<bool> _running;
        std::shared_ptr<Actor> _actor;
        LockCQueue<std::shared_ptr<NodeEvent>> _lockQueue;
    };

        class Actor
    {
    public:
        Actor() = default;

        ~Actor() = default;

    public:

        void Subscribe(Active *active, NodeSignal event);

        void UnSubscribe(Active *active, NodeSignal event);

        void Broadcast(std::shared_ptr<NodeEvent> event);

        void RegisterActor(std::shared_ptr<Active> active);

        void Start();

        void Stop();

    private:
        std::mutex _eventMutex;
        std::map<NodeSignal, std::set<Active *>> _subMap;
        std::vector<std::shared_ptr<Active>> _activeList;

    };
```

4.事件处理机制

```
///typedef std::function<void(std::shared_ptr<NodeEvent> event)> FuncNode;
```

```
///typedef std::function<FuncNode(std::shared_ptr<NodeEvent> event)> InterfaceFu

///订阅状态节点接口
///状态接口返回值是父级状态接口
typedef void (*FuncNode)(std::shared_ptr<NodeEvent> event);

typedef FuncNode(*InterfaceFuncNode)(std::shared_ptr<NodeEvent> event);

/**
*    当前状态的返回值必须为父级节点或nullptr
*    若返回父级至继续执行依次往上执行,否则认为此事件处理完毕,处理下一个消息事件
*/
     void StateMachine::Dispatch(std::shared_ptr<NodeEvent> e)
    {
        auto state = _current->_funNode;
        do
        {
            state = (InterfaceFuncNode) state(e);
        } while (state != nullptr);
    }
```